

CSE406: Cyber Security Sessional

Design Report

Packet sniffing attack and sniff http/telnet passwords

Group : 10

Group Members:

2105128 - Nakib Arman

2105143 - Fatin Ishrak Arian

1 Overview of the Project Idea

This project implements a passive packet-sniffing attack tool. The tool places the attacker’s network interface card (NIC) into promiscuous mode so that it receives every Ethernet frame arriving on its local network segment, not only frames addressed to it. The attacker’s machine acts purely as a listener — a “hop” that observes traffic — and does not modify, inject, redirect, or forward any traffic; no ARP spoofing or other active man-in-the-middle technique is used. Captured frames are parsed layer by layer (Ethernet → IP → TCP) using custom Python code built on raw sockets, and the reassembled application-layer payload is inspected for clear-text credentials carried by two legacy, unencrypted protocols: HTTP Basic Authentication and Telnet.

The underlying vulnerability is that HTTP (without TLS) and Telnet transmit authentication data as plain text inside the TCP payload. Any host able to observe the raw frames on the segment — for example a host on a hub, a mirrored switch port, or a wireless interface in monitor mode — can recover usernames and passwords without breaking any cryptography or interacting with the victim. This makes sniffing a purely passive, hard-to-detect attack.

Goal: capture live traffic between a victim client and a server, reconstruct the relevant packets in software, and automatically extract and display any HTTP or Telnet login credentials found in the captured payloads, along with per-packet metadata (source/destination MAC, IP, port, protocol, timestamp).

2 Network Topology, Components and Timing Diagram

2.1 Components

Component	Role
Victim Host	Runs an HTTP client and a Telnet client; logs into the server using plain-text credentials.
Server	Runs an HTTP server with Basic Authentication enabled and a Telnet daemon, both on the same LAN.
Switch / Hub	Connects all hosts to the same broadcast/collision domain, so the frames exchanged between the victim and the server are also visible to the attacker’s NIC.
Attacker Host	Runs the custom Python raw-socket sniffer with its NIC in promiscuous mode. Purely passive — no packets are transmitted toward the victim or server.

Promiscuous mode only helps if the attacker’s NIC actually receives the relevant frames. On a plain switched network, a switch forwards unicast frames only to the destination port, so the victim–server conversation would normally never reach the attacker’s port. To keep the topology realistic, all three hosts are placed on the same unmanaged hub segment (or, equivalently, on a switch port configured for mirroring), which matches the behavior of a bridged/internal VirtualBox network used for the demo.

2.2 Topology Diagram

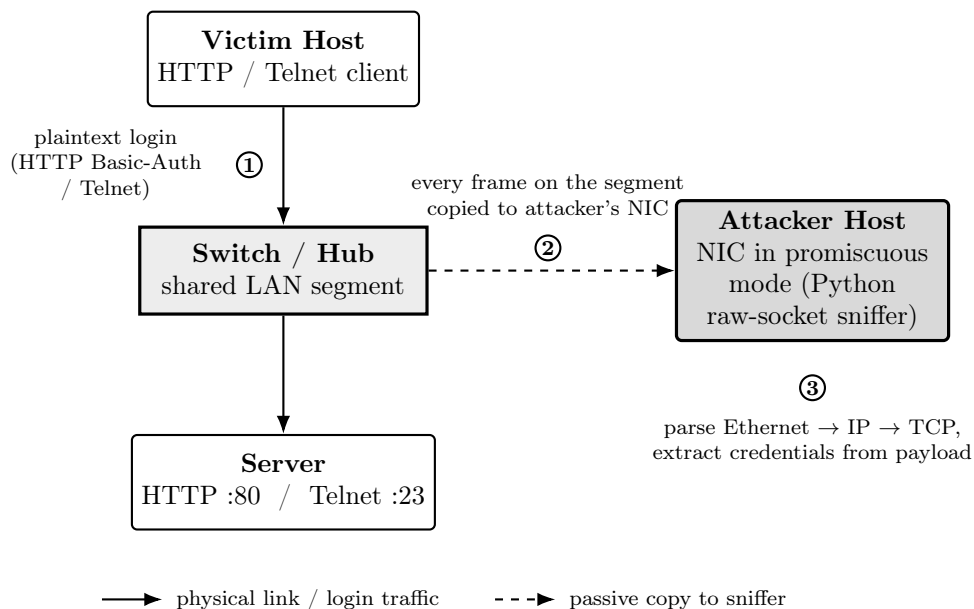


Figure 1: All hosts attach to the same hub/segment. The victim's plaintext login travels victim → hub → server; because a hub repeats every frame to all ports, the attacker's promiscuous NIC receives a copy of that same traffic (dashed) without ever transmitting anything back.

2.3 Timing Diagram

The timing diagram below contrasts the normal protocol timeline (victim establishing a TCP connection and logging in to the server) with the attacker's concurrent, passive observation of the same frames. Because the attacker only reads frames traversing the shared medium, its activity is not part of the protocol's control flow — the handshake and login proceed identically whether or not the sniffer is running.

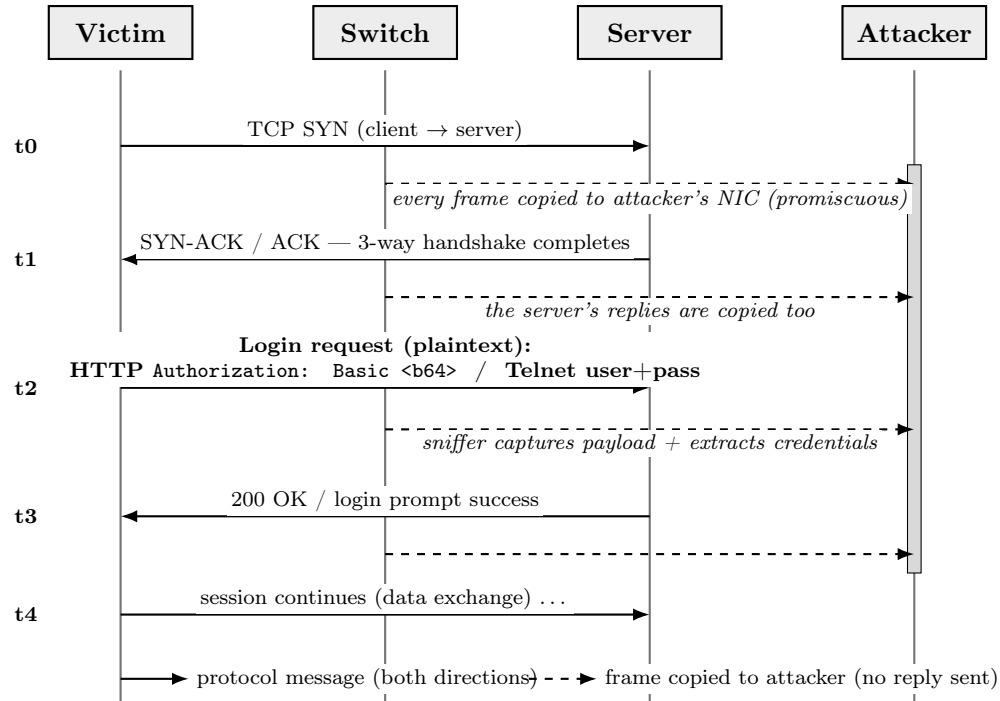


Figure 2: The victim-server exchange (solid) is unaffected by the sniffer. Because a hub repeats every frame to all ports, the attacker's promiscuous NIC receives a copy of *every* frame in *both* directions (dashed) — requests and responses alike — yet transmits nothing back. The credential-bearing frame is the plaintext login at t2; the shaded bar shows the NIC listening throughout.

3 Packet and Frame Structure

The sniffer parses three nested headers before reaching the credential-bearing payload. All offsets below are used directly when unpacking raw bytes with Python's `struct` module.

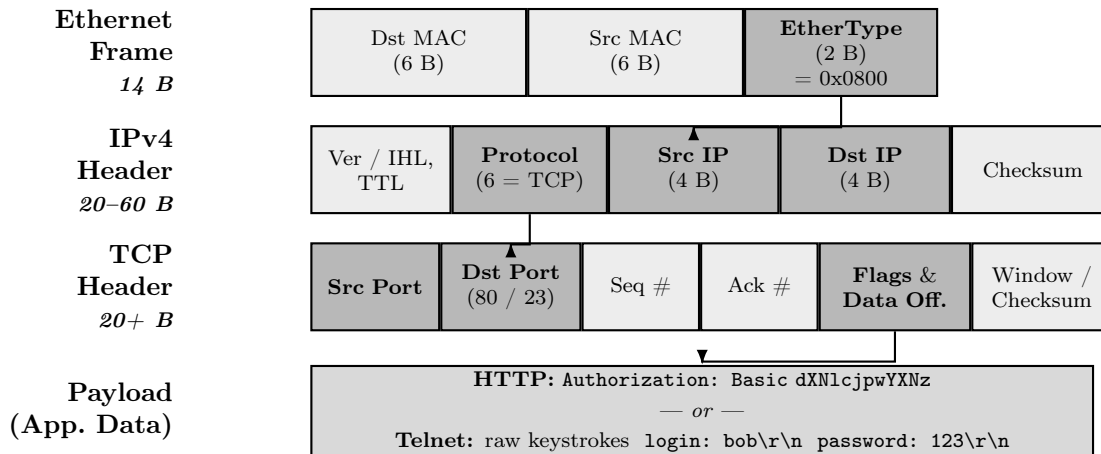


Figure 3: Layered frame structure the sniffer decodes, from the Ethernet header down to the application payload. Shaded fields are the ones the sniffer actually reads to route parsing and locate credentials; arrows show the top-down parse order (Ethernet → IP → TCP → Payload).

3.1 Fields Used by the Sniffer

Layer	Field(s) Used	Purpose in the Attack
Ethernet (14 B)	Dst MAC, Src MAC, EtherType	Confirms the frame carries an IPv4 payload (EtherType 0x0800) before further parsing.
IPv4 Header (20-60 B)	Version/IHL, Protocol, Src IP, Dst IP	Identifies the conversation endpoints and confirms Protocol = 6 (TCP) before handing off to TCP parsing.
TCP Header (20+ B)	Src Port, Dst Port, Flags, Data Offset	Dst/Src Port = 80 selects HTTP traffic; Port = 23 selects Telnet traffic. Data Offset locates where the header ends and the payload begins.
Application Payload	Raw bytes after the TCP header	Searched for Authorization: Basic (HTTP) and for Telnet login/password prompts followed by the victim's keystrokes.

No header field is modified by the attacker at any point — the tool only reads bytes off the wire. This distinguishes a sniffing attack from active attacks such as ARP poisoning or TCP reset, where the attacker crafts and transmits forged frames.

4 Implementation Plan

4.1 Technology Choices

- **Language:** Python 3, using `AF_PACKET` raw sockets (Linux) to capture frames directly from the data link layer.
- All Ethernet / IPv4 / TCP parsing is done manually with the `struct` module, following the header layouts of RFC 894 (Ethernet II), RFC 791 (IPv4) and RFC 793 (TCP).
- Test environment: two VMs (victim, server) and the attacker VM on the same internal/bridged network, running a minimal HTTP server with Basic Auth and a Telnet daemon.

4.2 Development Steps

1. **Raw capture:** open the raw socket, enable promiscuous mode, and confirm frames are arriving by comparing packet counts against Wireshark running on the same interface.
2. **Header parsing:** implement and test Ethernet, IPv4 and TCP unpacking; validate parsed fields (IPs, ports) against Wireshark's decode of the same frames.
3. **HTTP credential extraction:** generate a Basic-Auth HTTP login from the victim VM; confirm the sniffer isolates and base64-decodes the `Authorization` header correctly.
4. **Telnet credential extraction:** generate a Telnet login from the victim VM; implement per-connection buffering so a username/password split across several packets (Telnet often sends one TCP segment per keystroke) is reassembled correctly.
5. **Output, logging and validation:** print a live console table and a session log (timestamp, src, dst, protocol, recovered credential); repeat captures and cross-check every field against Wireshark before the demo.

5 Expected Outcome of a Successful Attack

A successful run produces, on the attacker's console/log, the exact username and password entered by the victim on the HTTP and Telnet sessions — recovered purely from passively observed frames, with zero packets sent by the attacker toward the victim or server. Concretely, we expect:

- A live listing of all TCP frames destined to ports 80 and 23 crossing the segment, with source/destination IP, MAC, and timestamp — matching what Wireshark reports for the same capture window.
- For the HTTP login: the correctly base64-decoded `username:password` pair extracted from the `Authorization: Basic` header.
- For the Telnet login: the reconstructed clear-text username and password, correctly reassembled from the individual per-keystroke TCP segments.

Success is judged against two criteria: (1) **correctness** — recovered credentials exactly match what the victim typed, and (2) **non-interference** — victim and server behavior/timing is unaffected by the presence of the sniffer, confirming the attack is fully passive.

6 Defense Idea

Sniffing succeeds only because HTTP and Telnet send credentials as clear text; the direct and complete defense is protocol-level encryption rather than trying to detect the sniffer itself, since a purely passive listener generates no network traffic of its own and is inherently hard to detect. Replacing HTTP with HTTPS (TLS) and Telnet with SSH makes the payload captured by any sniffer unreadable ciphertext, defeating this attack regardless of whether the attacker can still observe the frames on the wire. On the network side, restricting or disabling port mirroring and enabling switch port security limits how easily an attacker can arrange to receive a copy of someone else's traffic in the first place.